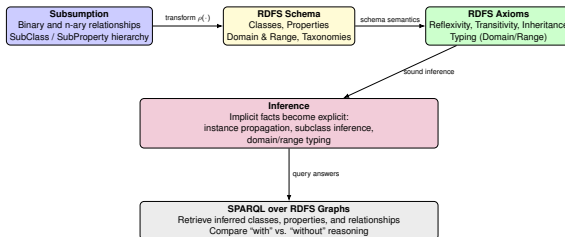


RDFS and Reasoning(Lecture 5)



From Schema to Inference: RDFS



From conceptual relationships → to RDFS schema → to logical axioms → to machine reasoning.

Agenda

■ 1. Semantic Data Modeling — Beyond EER

- Subsumption of Binary and n -ary Relationships
- Why subsumption matters for semantic schemas

■ 2. RDF Schema (RDFS)

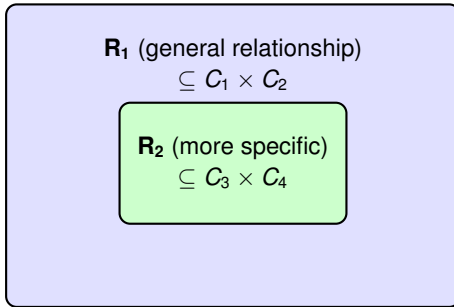
- Axioms and Semantics of RDFS [1]
- SubClassOf, SubPropertyOf, Domain, Range
- What machines can infer from an RDFS graph

■ 3. Reasoning in Practice

- SPARQL with and without inference
- **Closed World vs. Open World Assumption** (CWA vs. OWA)

From conceptual subsumption → to RDFS schemas → to machine reasoning.

Intuition Behind Subsumption



Key Idea

R_2 is completely contained inside R_1 .
Every pair in R_2 must also be a pair in R_1 .

Subsumption of Binary Relationship Types (1/2)

Idea. Sometimes a relationship is a *more specific version* of another relationship. In that case, the specific one is said to be a **sub-relationship** of the general one.

Formal Definition

Given two binary relationships:

$$R_1 \subseteq C_1 \times C_2 \quad \text{and} \quad R_2 \subseteq C_3 \times C_4$$

We say that:

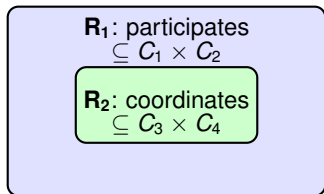
R_2 is a sub-relationship of R_1

iff:

$$R_2 \subseteq R_1$$

- Every pair in R_2 must also be a pair in R_1 .

Subsumption of Binary Relationship Types (2/2)



Every instance of `coordinates` is also an instance of `participates`.

Intuition:

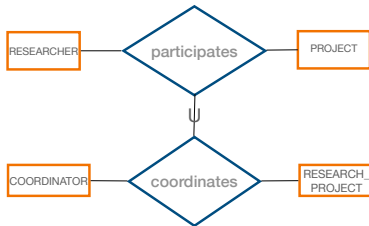
- Both R_1 and R_2 connect pairs of entities.
- R_2 is **more specific**—it applies to fewer, more constrained cases.
- $\Rightarrow$ All pairs in R_2 are also pairs in R_1 .

Example:

- `participates: researcher` $\rightarrow$ `project`
- `coordinates: coordinator` $\rightarrow$ `research project`
- Every coordinator participates in the project they coordinate.

This structure will later become an **RDFS subPropertyOf** relationship.

Examples- Subsumption of Binary Relationship Type

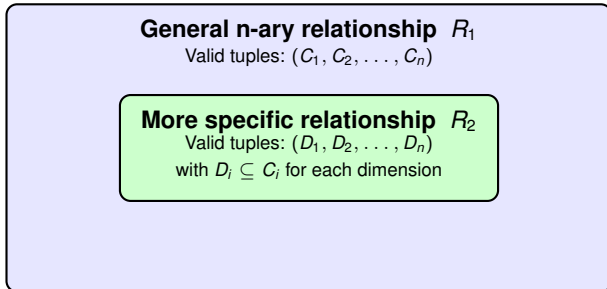


Explanation:

- participates and coordinates are both binary relationships.
- coordinates is a **sub-relationship** of participates.
- participates $\subseteq \text{RESEARCH} \times \text{PROJECT}$.
- coordinates $\subseteq \text{COORDINATOR} \times \text{RESEARCH_PROJECT}$.

Intuition: Subsumption of n -ary Relationship Types

An n -ary relationship is a set of n -tuples. If every tuple allowed by R_2 also satisfies the conditions of R_1 , then R_2 is a **more specific version** of R_1 .



- R_2 “fits inside” R_1 because it restricts each position of the n -tuple.
- A tuple in R_2 automatically satisfies the conditions of R_1 .
- Subsumption generalizes subclassing: instead of one element being more specific, **an entire tuple pattern is more specific**.

Subsumption of n -ary Relationship Types (1/2)

A relationship involving several entities (an n -tuple) can be a *more specific version* of another n -ary relationship. In that case, the specific one is a **sub-relationship** of the general one.

Formal Definition

Let:

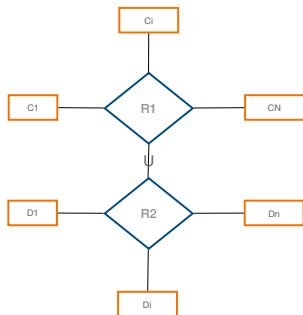
$$R_1 \subseteq C_1 \times C_2 \times \cdots \times C_n \quad \text{and} \quad R_2 \subseteq D_1 \times D_2 \times \cdots \times D_n$$

We say that R_2 is a sub-relationship of R_1 iff $R_2 \subseteq R_1$

- Each tuple in R_2 is simultaneously a valid tuple of R_1 .
- R_2 adds *more constraints* (e.g., more specific roles or types).
- This mirrors subclass behavior but for tuples of size $n \geq 3$.

Subsumption of n-ary Relationship Types (2/2)

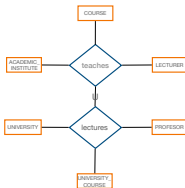
Explanation:



- R_1 and R_2 are both n -ary relationships, each connecting three entities.
- R_2 is a **sub-relationship** of R_1 , meaning every tuple in R_2 is also present in R_1 .
- $R_1 \subseteq C_1 \times \dots \times C_i \times \dots \times C_n$: Each tuple in R_1 connects instances from $C_1, \dots, C_i, \dots, C_n$.
- $R_2 \subseteq D_1 \times \dots \times D_i \times \dots \times D_n$: Each tuple in R_2 connects instances from $D_1, \dots, D_i, \dots, D_n$.
- Since $R_2 \subseteq R_1$, all tuples in R_2 are also contained in R_1 .

Example of n -ary Relationship Transformation (3/4)

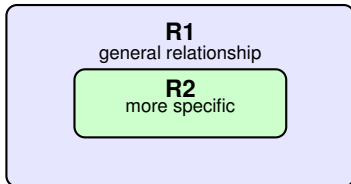
Explanation:



- **teaches**: Connects LECTURER, COURSE, and ACADEMIC_INSTITUTE.
- **lectures**: Connects PROFESSOR, UNIVERSITY_COURSE, and UNIVERSITY.
- The **subsumption** indicates that teaches subsumes lectures, meaning every instance of lectures is also an instance of teaches.

From Subsumption to RDFS: Binary Relationships

If a binary relationship R_2 is a *more specific* version of R_1 (i.e., every pair in R_2 also appears in R_1), then in RDFS this becomes a **subproperty** relation.



In RDFS: R_2 `rdfs:subPropertyOf` R_1

Transformation Rule.

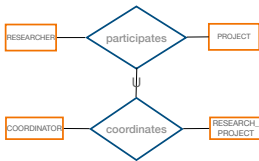
- Each entity type becomes an `rdfs:Class`.
- Each relationship becomes an `rdf:Property`.
- Subsumption becomes `rdfs:subPropertyOf`.

```
ex:R1 a rdf:Property ;  
      rdfs:domain ex:C1 ;  
      rdfs:range  ex:C2 .  
ex:R2 a rdf:Property ;  
      rdfs:domain ex:C3 ;  
      rdfs:range  ex:C4 .  
ex:R2 rdfs:subPropertyOf ex:R1 .
```

Meaning: any fact using R_2 also holds for R_1 .

Example- Subsumption of Binary Relationship Types

Representation in RDFS:



```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix ex: <http://example.org/> .

ex:RESEARCHER          a          rdfs:Class.
ex:PROJECT              a          rdfs:Class.
ex:COORDINATOR          a          rdfs:Class.
ex:ex:RESEARCH_PROJECT a          rdfs:Class.
ex:participates         a          rdf:Property;
                        rdfs:domain ex:RESEARCHER;
                        rdfs:range  ex:PROJECT.

ex:coordinates          a          rdf:Property;
                        rdfs:domain ex:COORDINATOR;
                        rdfs:range  ex:RESEARCH_PROJECT.

ex:coordinates          rdfs:subPropertyOf ex:participates.
```

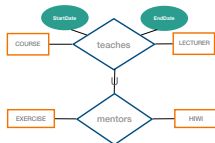
Subsumption of Relationships with Attributes (1/3)

When a relationship has attributes, it can no longer be modeled as an RDF property (only subject–object). Instead, it becomes an **RDFS class** with properties for its roles and attributes. If R_2 is a more specific relationship than R_1 , then in RDFS we express this as:

$$R_2 \text{ rdfs:subClassOf } R_1.$$

Subsumption of Relationships with Attributes (3/3)

Representation in RDFS:



```

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix ex: <http://example.org/> .

ex:COURSE                a                rdfs:Class.
ex:LECTURER              a                rdfs:Class.
ex:EXERCISE              a                rdfs:Class.
ex:HIWI                  a                rdfs:Class.
ex:teaches               a                rdfs:Class.
ex:mentors               a                rdfs:Class.
ex:teachesCOURSE         a                rdf:Property;
                        rdfs:domain      ex:teaches;
                        rdfs:range      ex:COURSE.
ex:teachesLECTURER      a                rdf:Property;
                        rdfs:domain      ex:teaches;
                        rdfs:range      ex:LECTURER.
ex:mentorsEXERCISE      a                rdf:Property;
                        rdfs:domain      ex:mentors;
                        rdfs:range      ex:EXERCISE.
ex:mentorsHIWI          a                rdf:Property;
                        rdfs:domain      ex:mentors;
                        rdfs:range      ex:HIWI.
ex:mentors               rdfs:subClassOf ex:teaches.
  
```

Subsumption of n -ary Relationships to RDFS (1/3)

Idea. An n -ary relationship becomes a **class** in RDFS (reification), because attributes and more than two roles cannot be expressed with a simple RDF property. If R_2 is more specific than R_1 , we express this as:

$$R_2 \text{ rdfs:subClassOf } R_1$$

From n -ary Subsumption to RDFS (2/3)

RDFS Transformation Template

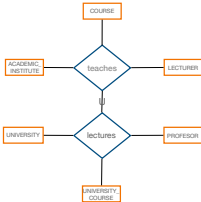
```
# Relationship classes
ex:R1 a rdfs:Class .
ex:R2 a rdfs:Class .

# Participant types
ex:C1 a rdfs:Class .      ...      ex:Cn a rdfs:Class .
ex:D1 a rdfs:Class .      ...      ex:Dn a rdfs:Class .

# Roles of R1
ex:R1_C1 a rdf:Property ;
    rdfs:domain ex:R1 ;
    rdfs:range  ex:C1 .
...
ex:R1_Cn a rdf:Property ;
    rdfs:domain ex:R1 ;
    rdfs:range  ex:Cn .

# Roles of R2
ex:R2_D1 a rdf:Property ;
    rdfs:domain ex:R2 ;
    rdfs:range  ex:D1 .
...
ex:R2_Dn a rdf:Property ;
```

Example (3/3)



R2 (lectures) is a special case of R1 (teaches)

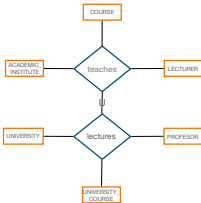
Relationship classes

```
ex:teaches a rdfs:Class .
```

```
ex:lectures a rdfs:Class ;
```

```
  rdfs:subClassOf ex:teaches .
```

Example (3/3)



R2 (lectures) is a special case of R1 (teaches)

Relationship classes

```
ex:teaches a rdfs:Class .
```

```
ex:lectures a rdfs:Class ;
  rdfs:subClassOf ex:teaches .
```

Roles of teaches

```
ex:teaches_LECTURER a rdfs:Property ;
  rdfs:domain ex:teaches ;
  rdfs:range ex:LECTURER .
```

ex:teaches_COURSE

```
a rdfs:Property ;
  rdfs:domain ex:teaches ;
  rdfs:range ex:COURSE .
```

ex:teaches_INSTITUTE

```
a rdfs:Property ;
  rdfs:domain ex:teaches ;
  rdfs:range ex:ACADEMIC_INSTITUTE .
```

Roles of lectures

```
ex:lectures_PROFESSOR a rdfs:Property ;
  rdfs:domain ex:lectures ;
  rdfs:range ex:PROFESSOR .
```

ex:lectures_UCOURSE

```
a rdfs:Property ;
  rdfs:domain ex:lectures ;
  rdfs:range ex:UNIVERSITY_COURSE .
```

ex:lectures_UNIVERSITY

```
a rdfs:Property ;
  rdfs:domain ex:lectures ;
  rdfs:range ex:UNIVERSITY .
```

RDF and RDFS Classes

More RDF/RDFS Properties

Structural Classes

- `rdfs:Resource`
The class of everything.
- `rdfs:Class`
The class of all classes.
- `rdf:Property`
The class of all properties.

Literals & Statements

- `rdfs:Literal`
Strings, numbers, booleans.
- `rdfs:Datatype`
Datatypes such as `xsd:string`.
- `rdf:Statement`
Reified triple.

These classes define the vocabulary for schema and data.

Core RDF and RDFS Properties

How RDF Schema Describes Structure

Typing & Hierarchy

- `rdf:type`
Instance → class
- `rdfs:subClassOf`
Class hierarchy
- `rdfs:subPropertyOf`
Property hierarchy

Schema Semantics

- `rdfs:domain`
Allowed subject type
- `rdfs:range`
Allowed object type

Documentation

- `rdfs:label` – readable name
- `rdfs:comment` – description
- `rdfs:seeAlso`, `rdfs:isDefinedBy` – references

These properties enable schema definition, inference, and documentation.

How Semantic Models Infer New Knowledge

Understand how **RDFS axioms** and the **Open World Assumption (OWA)** allow machines to infer **implicit knowledge** from *explicit data*.

Explicit Data

ex:a ex:P ex:b
ex:P rdfs:domain ex:C1
ex:P rdfs:range ex:C2

Given facts

Axioms + OWA

Typing rules
Subclass rules
Subproperty rules
Open World Assumption

Reasoning engine

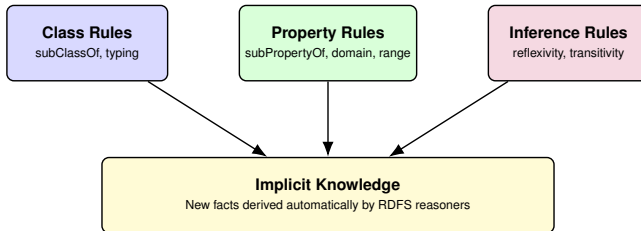
Inferred Knowledge

ex:a rdf:type ex:C1
ex:b rdf:type ex:C2
(and more)

Automatically derived

Semantic models: explicit facts + logic $\Rightarrow$ richer knowledge

Axiomatic System of RDFS



RDFS axioms combine *schema knowledge* (classes + properties) with *logic rules* to produce conclusions beyond what is explicitly stated in the graph.

Axiomatic System of RDFS

Axiomatic System of RDFS:

- provides a formal foundation for reasoning within RDF Schema.
- defines a set of logical rules and axioms that specify how classes, properties, and relationships interact in an RDFS graph.
- allows machines to derive implicit knowledge from explicit facts.

Axiomatic System of RDFS

Axiomatic System of RDFS:

- provides a formal foundation for reasoning within RDF Schema.
- defines a set of logical rules and axioms that specify how classes, properties, and relationships interact in an RDFS graph.
- allows machines to derive implicit knowledge from explicit facts.

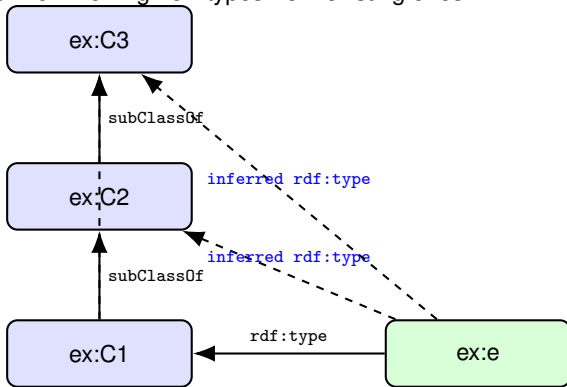
Key Components of the RDFS Axiomatic System:

- **Classes and Subclass Relationships:** Defines rules for subclassing (e.g., `rdfs:subClassOf`).
- **Properties and Subproperty Relationships:** Specifies how properties relate to each other and to classes (e.g., `rdfs:domain` and `rdfs:range`).
- **Inference Rules:** Logical rules that allow the deduction of new facts based on existing ones (e.g., transitivity and reflexivity in subclass relationships).

RDFS SubClass Axioms

What do the RDFS SubClass axioms capture?

- They define the **logical behavior of class hierarchies**.
- They allow for inferring new types from existing ones.



SubClass axioms explain: reflexivity, transitivity, and type propagation.

RDFS Axioms (1/4)

Group A (SubClass)

- 1. REFLEXIVITY:** IF (ex:C, a, rdfs:Class)
THEN (ex:C, rdfs:subClassOf, ex:C)
- 2. TRANSITIVITY:** IF (ex:C1, rdfs:subClassOf, ex:C2)
AND (ex:C2, rdfs:subClassOf, ex:C3)
THEN (ex:C1, rdfs:subClassOf, ex:C3)
- 3. IF** (ex:C1, rdfs:subClassOf, ex:C2)
AND (ex:e, a, ex:C1)
THEN (ex:e, a, ex:C2)

Example (1/3)

1. Reflexivity Example:

- **Axiom A1:** For any class `ex:Person`, it holds that `ex:Person rdfs:subClassOf ex:Person`.
- **Explanation:** This axiom states that every class is a subclass of itself.
- **Example:** `(ex:Person, a, rdfs:Class)` implies `(ex:Person, rdfs:subClassOf, ex:Person)`.

Example (2/3)

2. Transitivity Example:

- **Axiom A2:** If `ex:Lecturer rdfs:subClassOf ex:Person` and `ex:Professor rdfs:subClassOf ex:Lecturer`, then `ex:Professor rdfs:subClassOf ex:Person`.
- **Explanation:** This allows inference through subclass relationships.
- **Example:** (`ex:Professor, rdfs:subClassOf, ex:Lecturer`) and (`ex:Lecturer, rdfs:subClassOf, ex:Person`) imply (`ex:Professor, rdfs:subClassOf, ex:Person`).

Example (3/3)

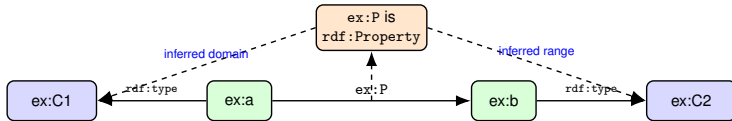
3. Inheritance Example:

- **Axiom A3:** If `ex:Professor` `rdfs:subClassOf` `ex:Lecturer` and `ex:Smith` is an instance of `ex:Professor`, then `ex:Smith` is also an instance of `ex:Lecturer`.
- **Explanation:** Instances of a subclass are also instances of its superclass.
- **Example:** (`ex:Smith`, `a`, `ex:Professor`) and (`ex:Professor`, `rdfs:subClassOf`, `ex:Lecturer`) imply (`ex:Smith`, `a`, `ex:Lecturer`).

RDFS Property Axioms

What do the RDFS Property Axioms ensure?

- They allow to inferring the **type of a property** from its usage.
- They let RDFS derive implicit **domain** and **range** information.



From one triple, RDFS can infer the property's type, domain, and range.

RDFS Axioms (2/4)

Group B (Properties)

1. **IF** (ex:a, ex:P, ex:b)
 THEN (ex:P, a, rdf:Property)
2. **IF** (ex:a, ex:P, ex:b)
 AND (ex:a, a, ex:C1)
 THEN (ex:P, rdfs:domain, ex:C1)
3. **IF** (ex:a, ex:P, ex:b)
 AND (ex:b, a, ex:C2)
 THEN (ex:P, rdfs:range, ex:C2)

Example (1/3)

Axiom 1: Property Existence

- **Axiom B1:** If there exists a statement (`ex:JohnSmith`, `ex:hasName`, `"John"`), then `ex:hasName` is recognized as an `rdf:Property`.
- **Explanation:** This axiom asserts that if a property is used in a triple, it is automatically inferred to be an `rdf:Property`.
- **Example:** (`ex:JohnSmith`, `ex:hasName`, `"John"`) implies (`ex:hasName`, `a`, `rdf:Property`).

Example (2/3)

Axiom B2: Property Domain

- **Axiom:** If $(\text{ex:JohnSmith}, \text{ex:hasName}, \text{"John"})$ and ex:JohnSmith is an instance of ex:Person , then ex:hasName has ex:Person as its domain.
- **Explanation:** This axiom specifies that if a property links an entity of a certain type, that type is the domain of the property.
- **Example:** $(\text{ex:JohnSmith}, a, \text{ex:Person})$ and $(\text{ex:JohnSmith}, \text{ex:hasName}, \text{"John"})$ imply $(\text{ex:hasName}, \text{rdfs:domain}, \text{ex:Person})$.

Example (3/3)

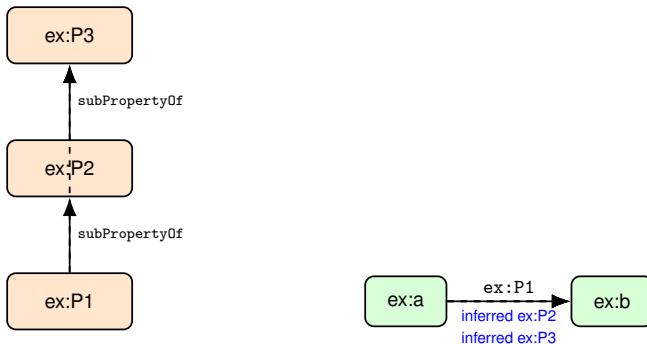
Axiom B3: Property Range

- **Axiom:** If $(\text{ex:JohnSmith}, \text{ex:hasAddress}, \text{ex:Address1})$ and ex:Address1 is an instance of ex:Address , then ex:hasAddress has ex:Address as its range.
- **Explanation:** This axiom asserts that if a property links to an entity of a certain type, that type is the range of the property.
- **Example:** $(\text{ex:Address1}, a, \text{ex:Address})$ and $(\text{ex:JohnSmith}, \text{ex:hasAddress}, \text{ex:Address1})$ imply $(\text{ex:hasAddress}, \text{rdfs:range}, \text{ex:Address})$.

RDFS SubProperty Axioms

What do SubProperty axioms capture?

- They define how properties inherit meaning from more general properties.
- They ensure that property hierarchies behave like class hierarchies.



If P1 is more specific than P2, any fact using P1 also holds for P2 (and its superproperties).

RDFS Axioms (3/4)

Group C (SubProperties)

1. **REFLEXIVITY:** IF (ex:P, a, rdf:Property)
THEN (ex:P, rdfs:subPropertyOf, ex:P)
2. **TRANSITIVITY:** IF (ex:P1, rdfs:subPropertyOf, ex:P2)
AND (ex:P2, rdfs:subPropertyOf, ex:P3)
THEN (ex:P1, rdfs:subPropertyOf, ex:P3)
3. IF (ex:P1, rdfs:subPropertyOf, ex:P2)
AND (ex:a, ex:P1, ex:b)
THEN (ex:a, ex:P2, ex:b)

Example (1/3)

Axiom 1: Reflexivity of SubPropertyOf

- **Axiom C1:** If `ex:teaches` is an `rdf:Property`, then it is a subproperty of itself.
- **Explanation:** This axiom states that every property is a subproperty of itself.
- **Example:** `(ex:teaches, a, rdf:Property)` implies `(ex:teaches, rdfs:subPropertyOf, ex:teaches)`.

Example (2/3)

Axiom C2: Transitivity of SubPropertyOf

- **Axiom:** If `ex:lectures` is a subproperty of `ex:teaches`, and `ex:teaches` is a subproperty of `ex:educates`, then `ex:lectures` is also a subproperty of `ex:educates`.
- **Explanation:** This axiom allows transitive inheritance through subproperties.
- **Example:** (`ex:lectures`, `rdfs:subPropertyOf`, `ex:teaches`) and (`ex:teaches`, `rdfs:subPropertyOf`, `ex:educates`) imply (`ex:lectures`, `rdfs:subPropertyOf`, `ex:educates`).

Example (3/3)

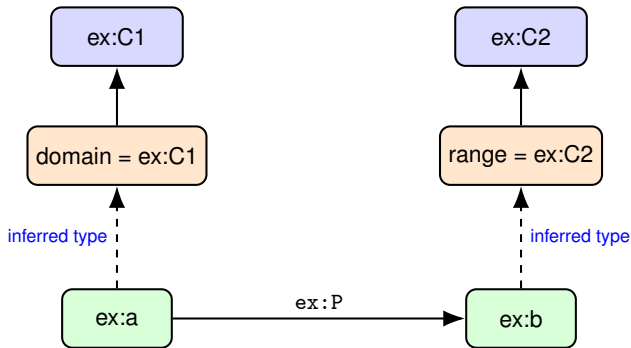
Axiom C3: Inheritance of Property Use

- **Axiom:** If `ex:lectures` is a subproperty of `ex:teaches` and `(ex:DrSmith, ex:lectures, ex:Course101)` holds, then `(ex:DrSmith, ex:teaches, ex:Course101)` also holds.
- **Explanation:** Instances using a subproperty imply the use of its parent property.
- **Example:** `(ex:DrSmith, ex:lectures, ex:Course101)` and `(ex:lectures, rdfs:subPropertyOf, ex:teaches)` imply `(ex:DrSmith, ex:teaches, ex:Course101)`.

Understanding RDFS Typing Axioms

What do Typing axioms enforce?

- They connect domain/range **schema constraints** with **instance data**.
- They ensure subjects and objects receive the **correct types**.
- They support automatic classification through inference.



RDFS Axioms (4/4)

Group D (Typing)

1. **IF** (ex:P, rdfs:domain, ex:C1)

AND (ex:a, ex:P, ex:b)

THEN (ex:a, a, ex:C1)

2. **IF** (ex:P, rdfs:range, ex:C2)

AND (ex:a, ex:P, ex:b)

THEN (ex:b, a, ex:C2)

Example (1/2)

Axiom 1: Domain Typing

- **Axiom D1:** If `ex:hasAdvisor` has a domain of `ex:Student` and `ex:Alice ex:hasAdvisor ex:DrSmith`, then `ex:Alice` is inferred to be a `Student`.
- **Explanation:** This axiom specifies that if an entity is using a property with a defined domain, that entity is an instance of the domain class.
- **Example:** (`ex:hasAdvisor`, `rdfs:domain`, `ex:Student`) and (`ex:Alice`, `ex:hasAdvisor`, `ex:DrSmith`) imply (`ex:Alice`, `a`, `ex:Student`).

Example (2/2)

Axiom D2: Range Typing

- **Axiom:** If `ex:hasAdvisor` has a range of `ex:Professor` and `ex:Alice ex:hasAdvisor ex:DrSmith`, then `ex:DrSmith` is inferred to be a `Professor`.
- **Explanation:** This axiom specifies that if a property links to an entity with a defined range, that entity is an instance of the range class.
- **Example:** (`ex:hasAdvisor`, `rdfs:range`, `ex:Professor`) and (`ex:Alice`, `ex:hasAdvisor`, `ex:DrSmith`) imply (`ex:DrSmith`, `a`, `ex:Professor`).

Intuition: Closed vs. Open World Assumptions

How do different systems interpret *missing information*?

Closed World Assumption (CWA)

*If it is not known to be true,
assume it is false.*

Databases, SQL, data warehouses

Missing row in a table = the fact does NOT hold.

Open World Assumption (OWA)

*If it is not known to be true,
assume nothing.
It may be true or false—we
simply don't know.*

RDF, RDFS, OWL, Knowledge Graphs

Missing fact in a KG = information is incomplete, not false.

CWA = “absence of evidence is evidence of absence” · OWA = “absence of evidence is not evidence of absence”

Assumptions for Unknown Values

Definition

The Closed World Assumption (CWA) assumes that what is unknown to be true is assumed to be false.

Assumptions for Unknown Values

Definition

The Closed World Assumption (CWA) assumes that what is unknown to be true is assumed to be false.

Definition

The Open World Assumption (OWA) assumes that what is not known to be true is simply unknown, not false.

Closed World Assumption (CWA)

Statements represented in a relational database

Coordinates Table:

Senior_Researcher_MD	Researcher_Project_MD
ex:JohnSmith ex:MarySmith ex:PhilippLange	ex:CLARIFY ex:CLARIFY ex:EUCLARIFY

Question: Does `ex:JohnSmith` coordinate `ex:EUCLARIFY`?

Answer: **NO** (According to CWA)

Explanation: Under the semantics of traditional databases, the answer is **NO** because there is no row in the relation Coordinates linking `ex:JohnSmith` to `ex:EUCLARIFY`.

Open World Assumption (OWA)

```
@prefix ex: <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
ex:Senior_Researcher_MD a rdfs:Class .
ex:Research_Project_MD a rdfs:Class .
ex:JohnSmith a ex:Senior_Researcher_MD .
ex:MarySmith a ex:Senior_Researcher_MD .
ex:PhilippLange a ex:Senior_Researcher_MD .
ex:CLARIFY a ex:Research_Project_MD .
ex:EUCLARIFY a ex:Research_Project_MD .
ex:coordinates a rdf:Property ;
    rdfs:domain ex:Senior_Researcher_MD ;
    rdfs:range ex:Research_Project_MD .
ex:JohnSmith ex:coordinates ex:CLARIFY .
ex:MarySmith ex:coordinates ex:CLARIFY .
ex:PhilippLange ex:coordinates ex:EUCLARIFY .
```

Open World Assumption (OWA)

```
@prefix ex: <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
ex:Senior_Researcher_MD a rdfs:Class .
ex:Research_Project_MD a rdfs:Class .
ex:JohnSmith a ex:Senior_Researcher_MD .
ex:MarySmith a ex:Senior_Researcher_MD .
ex:PhilippLange a ex:Senior_Researcher_MD .
ex:CLARIFY a ex:Research_Project_MD .
ex:EUCLARIFY a ex:Research_Project_MD .
ex:coordinates a rdf:Property ;
    rdfs:domain ex:Senior_Researcher_MD ;
    rdfs:range ex:Research_Project_MD .
ex:JohnSmith ex:coordinates ex:CLARIFY .
ex:MarySmith ex:coordinates ex:CLARIFY .
ex:PhilippLange ex:coordinates ex:EUCLARIFY .
```

Question: Does `ex:JohnSmith` coordinate `ex:EUCLARIFY`?

Answer: **UNKNOWN** (According to OWA)

Open World Assumption (OWA)

```
@prefix ex: <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
ex:Senior_Researcher_MD a rdfs:Class .
ex:Research_Project_MD a rdfs:Class .
ex:JohnSmith a ex:Senior_Researcher_MD .
ex:MarySmith a ex:Senior_Researcher_MD .
ex:PhilippLange a ex:Senior_Researcher_MD .
ex:CLARIFY a ex:Research_Project_MD .
ex:EUCLARIFY a ex:Research_Project_MD .
ex:coordinates a rdf:Property ;
    rdfs:domain ex:Senior_Researcher_MD ;
    rdfs:range ex:Research_Project_MD .
ex:JohnSmith ex:coordinates ex:CLARIFY .
ex:MarySmith ex:coordinates ex:CLARIFY .
ex:PhilippLange ex:coordinates ex:EUCLARIFY .
```

Question: Does `ex:JohnSmith` coordinate `ex:EUCLARIFY`?

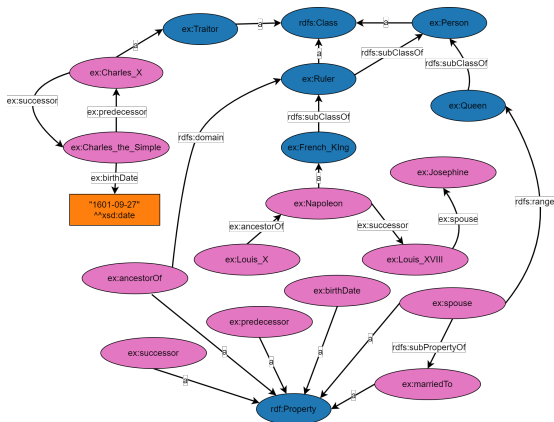
Answer: **UNKNOWN** (According to OWA)

Explanation: In RDFS graphs, the answer is **UNKNOWN**. The knowledge could be incomplete, so the relation between `ex:JohnSmith` and `ex:EUCLARIFY` might exist but is currently unrecorded.

Example: RDFS Axioms



- Demonstrates class hierarchies and relationships
- Will use this to explore RDFS Axioms



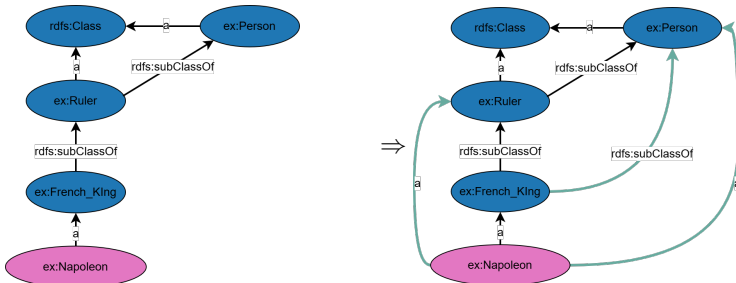
Example RDFS graph: French Emperor
Napoleon Bonaparte

Exercise

Participate in the survey at **Stud.IP**. Consider the following notation:

RDFS Axioms Summary		
Group A: SubClass		
Identifier	Title	Description
A1	Reflexivity	An entity is a subclass of itself.
A2	Transitivity	Subclass relationships are transitive.
A3	Inheritance	Instances of a subclass are also instances of the superclass.
Group B: Properties		
B1	Property Existence	Any property used implies the existence of the property.
B2	Property Domain	Domain restrictions on the property's subject.
B3	Property Range	Range restrictions on the property's object.
Group C: SubProperties		
C1	Reflexivity	A property is a subproperty of itself.
C2	Transitivity	Subproperty relationships are transitive.
C3	Inheritance of Property Use	If an instance uses a subproperty, it also uses the superproperty.
Group D: Typing		
D1	Domain Typing	Typing of the subject based on the property's domain.
D2	Range Typing	Typing of the object based on the property's range.

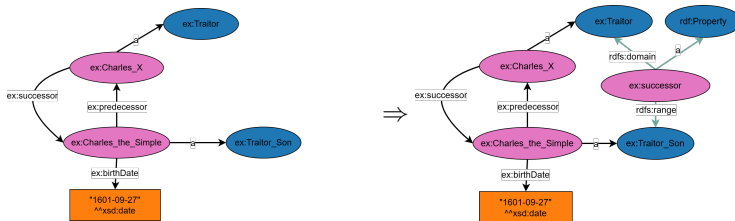
Example: Group A RDFS Axioms (1/4)



Axiom:

- **A2 (Transitivity) Example:** If `ex:French_King rdfs:subClassOf ex:Ruler` and `ex:Ruler rdfs:subClassOf ex:Person`, then `ex:French_King rdfs:subClassOf ex:Person`.
- **A3 (Inheritance) Example:** If `ex:French_King rdfs:subClassOf ex:Ruler` and `ex:Napoleon a ex:French_King`, then `ex:Napoleon a ex:Ruler`.

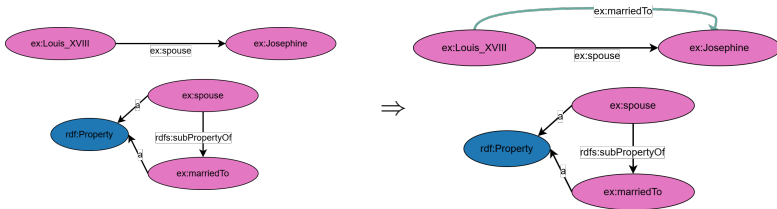
Group B RDFS Axioms (2/4)



Axiom:

- **B1 (Property Existence) Example:** If `ex:Charles_X ex:successor ex:Charles_the_Simple` then `ex:successor a rdf:Property`.
- **B2 (Property Domain) Example:** If `ex:Charles_X ex:successor ex:Charles_the_Simple` and `ex:Charles_X a ex:Traitor`, then `ex:successor rdfs:domain ex:Traitor`.
- **B3 (Property Range) Example:** If `ex:Charles_X ex:successor ex:Charles_the_Simple` and `ex:Charles_the_Simple a ex:Traitor_Son`, then `ex:successor rdfs:range ex:Traitor_Son`.

Example: Group C RDFS Axioms (3/4)

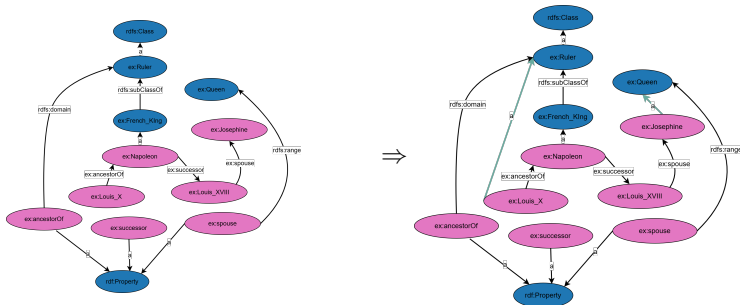


Axiom:

■ C3 (Inheritance) Example of Property Use:

If `ex:spouse rdfs:subPropertyOf ex:marriedTo` and `ex:Louis_XVIII ex:spouse ex:Josephine`, then
`ex:Louis_XVIII ex:marriedTo ex:Josephine`.

Group D RDFS Axioms (4/4)



Axiom:

- **D1 (Domain Typing) Example:** If `ex:ancestOf` `rdfs:domain` `ex:Ruler` and `ex:Louis_X` `ex:ancestOf` `ex:Napoleon`, then `ex:Louis_X` `a` `ex:Ruler`.
- **D2 (Range Typing) Example:** If `ex:spouse` `rdfs:range` `ex:Queen` and `ex:Louis_XVIII` `ex:spouse` `ex:Josephine`, then `ex:Josephine` `a` `ex:Queen`.

Exploring RDFS Graphs

SPARQL Queries

Access the KGs: a) FrenchRoyalty and b)
FrenchRoyaltyInference.

`https://labs.tib.eu/sdm/graphdb\_luh/sparql`

SPARQL Queries (1/3)

Query 1: Retrieve all the types to which `ex:Napoleon` belongs to.

SPARQL Query:

```
PREFIX ex: <http://example.org/>  
SELECT DISTINCT ?Class  
WHERE { ex:Napoleon a ?Class }
```

Query Results:

Without Reasoning

	Class
1	ex:French_King

SPARQL Queries (1/3)

Query 1: Retrieve all the types to which `ex:Napoleon` belongs to.
SPARQL Query:

```
PREFIX ex: <http://example.org/>  
SELECT DISTINCT ?Class  
WHERE { ex:Napoleon a ?Class }
```

Query Results:

Without Reasoning

	Class
1	ex:French_King

With RDFS Reasoning

	Class
1	ex:Person
2	ex:Ruler
3	ex:French_King

SPARQL Queries (2/3)

Query 2: To whom is married ex:Louis_XVIII?.

SPARQL Query:

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?spouse
WHERE {ex:Louis_XVIII ex:marriedTo ?spouse }
```

Query Results:

Without Reasoning

spouse

No data available in table

SPARQL Queries (2/3)

Query 2: To whom is married ex:Louis_XVIII?.

SPARQL Query:

```
PREFIX ex: <http://example.org/>  
SELECT DISTINCT ?spouse  
WHERE {ex:Louis_XVIII ex:marriedTo ?spouse }
```

Query Results:

Without Reasoning

spouse	
No data available in table	

With RDFS Reasoning

spouse	
1	ex:Josephine

SPARQL Queries (3/3)

Query 3: Who are persons?.

SPARQL Query:

```
PREFIX ex: <http://example.org/>  
SELECT DISTINCT ?person  
WHERE {?person a ex:Person}
```

Query Results:

Without Reasoning

person

No data available in table

SPARQL Queries (3/3)

Query 3: Who are persons?.

SPARQL Query:

```
PREFIX ex: <http://example.org/>  
SELECT DISTINCT ?person  
WHERE {?person a ex:Person}
```

Query Results:

Without Reasoning

	person
No data available in table	

With RDFS Reasoning

	person
1	ex:Charles_X
2	ex:Charles_the_Simple
3	ex:Louis_X
4	ex:Louis_XVIII
5	ex:Josephine
6	ex:Napoleon

Lecture Summary

Traditional Data Models (e.g., Relational)	RDF and RDFS
Closed World Assumption Data and Knowledge are complete	Open World Assumption Data and Knowledge may be incomplete
Integrity Constraints define the correctness of a database	RDFS Axioms and integrity constraints are used to deduce missing relationships

Lecture Summary

Traditional Data Models (e.g., Relational)	RDF and RDFS
Closed World Assumption Data and Knowledge are complete	Open World Assumption Data and Knowledge may be incomplete
Integrity Constraints define the correctness of a database	RDFS Axioms and integrity constraints are used to deduce missing relationships

Key Takeaways:

- RDF/RDFS enable an Open World Assumption, allowing incomplete knowledge representation.
- Deduction of new relationships through RDFS axioms enhances data integration and interoperability.
- Traditional data models prioritize data completeness and strict integrity constraints.

References I



C. Gutierrez, C. A. Hurtado, A. O. Mendelzon, and J. Pérez.
Foundations of semantic web databases.
J. Comput. Syst. Sci., 77(3):520–541, 2011.